

Μεταγλωττιστές 2025-26

Συντακτική ανάλυση top-down LL(1)

Συντακτική ανάλυση top-down LL(1)

- Για μια σημαντική υποκατηγορία context-free γραμματικών με πολυπλοκότητα $O(n)$
 - Σάρωση εισόδου από αριστερά προς τα δεξιά (left-to-right)
 - Αριστερότερες ακολουθίες παραγωγής (leftmost derivations)
 - Συμβουλευόμενοι ένα (το επόμενο) token εισόδου σε κάθε βήμα

Προϋποθέσεις για αριστερότερες παραγωγές (1)

- Οι κανόνες δεν πρέπει να έχουν **αριστερή αναδρομή**

Expr $\rightarrow$ **Expr** + Term

- Αλλιώς ο συντακτικός αναλυτής πέφτει σε **άπειρη αναδρομή**
- Απαλείφεται με αλγοριθμικό τρόπο, π.χ. ο κανόνας

$A \rightarrow A \alpha \mid \beta$

(όπου α, β ακολουθίες οποιωνδήποτε συμβόλων) γίνεται

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \varepsilon$

- **Προσοχή**: η απαλοιφή πρέπει να γίνει και σε περιπτώσεις **έμμεσης αναδρομής** (αλυσιδωτά, μετά από περισσότερα από ένα βήματα/κανόνες)

Προϋποθέσεις για αριστερότερες παραγωγές (2)

- Δεν πρέπει να υπάρχει **κοινός παράγοντας** στην αρχή του δεξιού μέρους ενός κανόνα, π.χ.

`Factor` $\rightarrow$ `id` | `id` [`ExprList`] | `id` (`ExprList`)

- Μετασχηματισμός με προσθήκη ενδιάμεσου κανόνα

`Factor` $\rightarrow$ `id` `Args`

`Args` $\rightarrow$ ϵ | [`ExprList`] | (`ExprList`)

Υλοποίηση συντακτικού αναλυτή LL(1)

- Από τη Θεωρία Υπολογισμού έχουμε ένα θεωρητικό εργαλείο
 - Pushdown Automaton (PDA)
 - Αυτόματο ενισχυμένο με μια στοίβα (stack)
 - Για κάθε μετάβαση το αυτόματο συμβουλεύεται το σύμβολο εισόδου και την κορυφή της στοίβας
 - Για να αποφασίσει ποια θα είναι η επόμενη κατάσταση
 - Και αν θα εξαχθεί ή θα αντικατασταθεί το στοιχείο στην κορυφή της στοίβας, και αν/ποια νέα στοιχεία θα εισαχθούν στη στοίβα

PDA και Context-free γραμματικές

- Υπάρχει πάντα ένα αυτόματο PDA **ισοδύναμο σε αναγνωριστική ισχύ** με μια γραμματική χωρίς συμφραζόμενα (CFG)
 - Αναγνωρίζει την **ίδια** γλώσσα
- Για τις γραμματικές LL(1) το αυτόματο PDA είναι **αιτιοκρατικό** (ντετερμινιστικό)
 - Είναι δυνατή το πολύ μια μετάβαση
 - Οι γραμματικές LL(1) εξ' ορισμού δεν μπορούν να είναι αμφίσημες

Πώς χρησιμοποιείται το αυτόματο PDA στη συντακτική ανάλυση

- Αρχικά στη στοίβα υπάρχει το **αρχικό** μη τερματικό σύμβολο της γραμματικής
- Το αυτόματο επαναλαμβάνει:
 - **predict**
 - Αν στην κορυφή της στοίβας βρίσκεται **μη τερματικό σύμβολο A** , τότε το A αντικαθίσταται στη στοίβα από το δεξιό μέρος ενός κατάλληλου κανόνα της γραμματικής, σύμφωνα και με το **επόμενο σύμβολο** στην είσοδο (η είσοδος **δεν καταναλώνεται**)
 - **match**
 - Αν στην κορυφή της στοίβας βρίσκεται **τερματικό σύμβολο a** , τότε, εφόσον το **επόμενο σύμβολο** εισόδου είναι ίδιο με το a, το a αφαιρείται από τη στοίβα (και από την είσοδο)
 - Σε κάθε άλλη περίπτωση παράγεται σφάλμα

Απλή (μη ρεαλιστική) περίπτωση LL(1)

- Έστω η γραμματική

$S \rightarrow a B$

$B \rightarrow b \mid a B b$

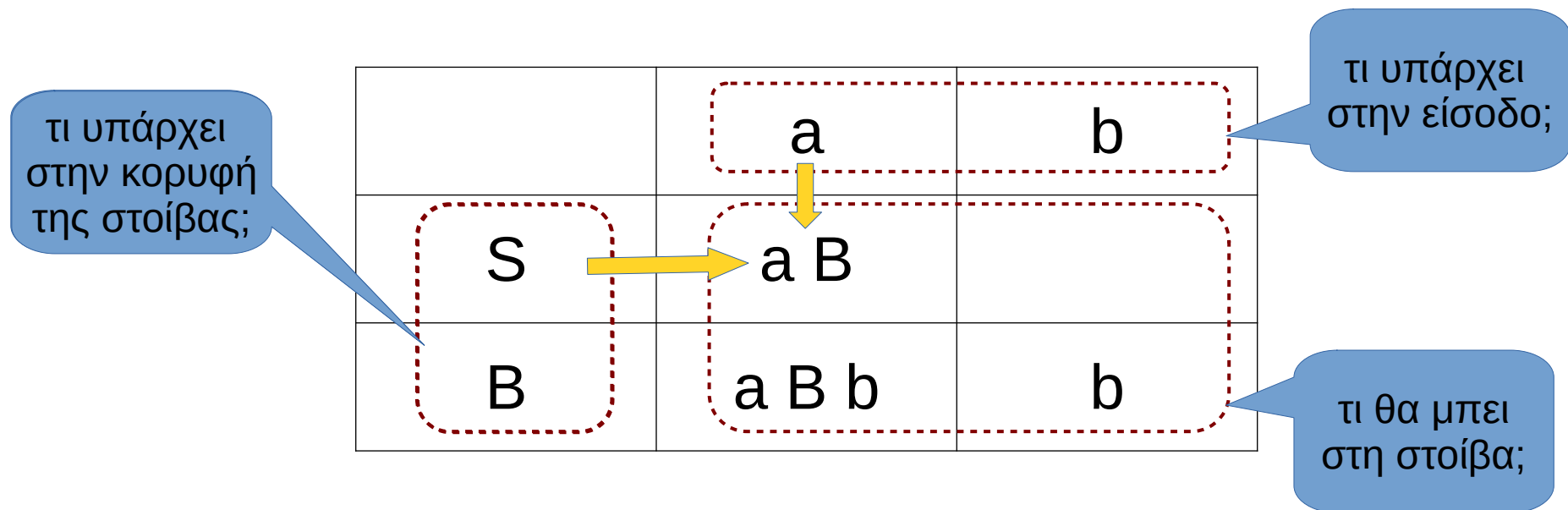
- Όλα τα δεξιά μέρη των κανόνων ξεκινούν με **τερματικό** σύμβολο

– **διαφορετικό** για τους εναλλακτικούς κανόνες του **ίδιου** μη τερματικού συμβόλου

- π.χ. $B \rightarrow b$ και $B \rightarrow a B b$

Πίνακας συντακτικής ανάλυσης

- Θεωρητικό εργαλείο, μέρος του αυτομάτου PDA
 - μπορεί επίσης να χρησιμοποιηθεί και σε έναν πραγματικό συντακτικό αναλυτή
 - επιλέγει τι θα μπει στη στοίβα στο βήμα **predict**



Π.χ. αν στην κορυφή της στοίβας είναι το **S** και το επόμενο σύμβολο στην είσοδο είναι το **a**, το S θα αντικατασταθεί στη στοίβα από το **a B**

	a	b
S	a B	
B	a B b	b

Είσοδος:

a a b b #

end-of-text (EOT)

κορυφή
στοίβας

S
#

Στοίβα

Σημ.: θεωρούμε ότι το αυτόματο έχει
μία και μοναδική κατάσταση, στην
οποία βρίσκεται πάντα

	a	b
S	a B	
B	a B b	b

“Predict”

Είσοδος:

a a b b #

S
#

Στοίβα

	a	b
S	a B	
B	a B b	b

“Predict”

Είσοδος:

a a b b #

a
B
#

Στοίβα

	a	b
S	a B	
B	a B b	b

“Match”

Είσοδος:

a a b b #

a
B
#

Στοίβα

	a	b
S	a B	
B	a B b	b

“Match”

Είσοδος:

a b b #

B
#

Στοίβα

	a	b
S	a B	
B	a B b	b

“Predict”

Είσοδος:

a b b #

B
#

Στοίβα

	a	b
S	a B	
B	a B b	b

“Predict”

Είσοδος:

a b b #

a
B
b
#

Στοίβα

	a	b
S	a B	
B	a B b	b

“Match”

Είσοδος:

a b b #

a
B
b
#

Στοίβα

	a	b
S	a B	
B	a B b	b

“Match”

Είσοδος:

b b #

B
b
#

Στοίβα

	a	b
S	a B	
B	a B b	b

“Predict”

Είσοδος:

b b #

B
b
#

Στοίβα

	a	b
S	a B	
B	a B b	b

“Predict”

Είσοδος:

b b #

b
b
#

Στοίβα

	a	b
S	a B	
B	a B b	b

“Match”

Είσοδος:

b b #

b
b
#

Στοίβα

	a	b
S	a B	
B	a B b	b

“Match”

Είσοδος:

b #

b
#

Στοίβα

	a	b
S	a B	
B	a B b	b

“Match”

Είσοδος:

b #

b
#

Στοίβα

	a	b
S	a B	
B	a B b	b

“Match”

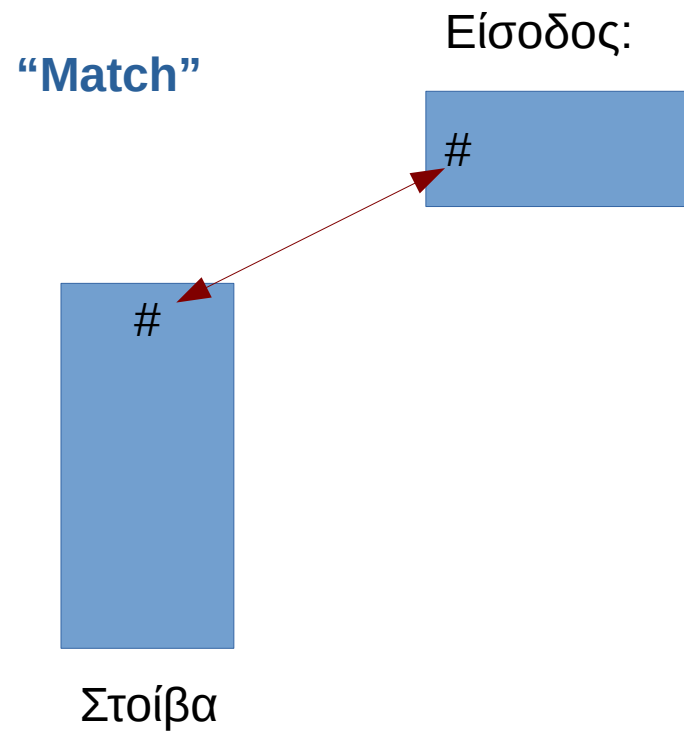
Είσοδος:

#

#

Στοίβα

	a	b
S	a B	
B	a B b	b



	a	b
S	a B	
B	a B b	b

Τερματισμός

Είσοδος:



Στοίβα

Κατασκευή συντακτικού αναλυτή LL(1)

- Θα μπορούσαμε να φτιάξουμε ένα πρόγραμμα που να **μιμείται** τη λειτουργία ενός PDA
- Συνήθως όμως υλοποιούμε τον συντακτικό αναλυτή ως ένα σύνολο **συναρτήσεων** που καλούν η μία την άλλη (ενδεχομένως και αναδρομικά)
 - Stack του PDA: υλοποιείται από τη **στοίβα κλήσης** των συναρτήσεων
 - Μπορούμε να κάνουμε σημασιολογική ανάλυση και να κατασκευάσουμε το AST μέσα στις συναρτήσεις αυτές
- Ο συντακτικός αναλυτής που κατασκευάζεται με τη μέθοδο αυτή ονομάζεται **«αναδρομικής κατάβασης»**
 - Recursive descent parser
 - Υλοποιεί και άλλους τύπους αναλυτών εκτός από LL(1)

Μέθοδος αναδρομικής κατάβασης

- Για κάθε **μη τερματικό σύμβολο** φτιάχνουμε **μια συνάρτηση**
 - Στο παράδειγμα της προηγούμενης γραμματικής

$S \rightarrow a B$

$B \rightarrow b \mid a B b$

θα υπάρχουν οι συναρτήσεις $S()$ και $B()$

Πρώτη καλείται η συνάρτηση του **αρχικού μη τερματικού συμβόλου** της γραμματικής, δηλαδή η $S()$

Μέθοδος αναδρομικής κατάβασης

- Κάθε συνάρτηση υλοποιεί όλους τους **κανόνες** που έχουν στο αριστερό μέρος το αντίστοιχο μη τερματικό σύμβολο, π.χ.
 - Η συνάρτηση **S ()** υλοποιεί τον κανόνα
$$S \rightarrow a B$$
 - Η συνάρτηση **B ()** υλοποιεί τους κανόνες
$$B \rightarrow b$$
$$B \rightarrow a B b$$

Μέθοδος αναδρομικής κατάβασης

- Υλοποίηση ενός κανόνα
 - Για το **δεξί μέρος** του κανόνα, από αριστερά προς τα δεξιά:
 - Για κάθε **τερματικό σύμβολο**, καλώ μια συνάρτηση **match()** η οποία, αν η είσοδος ταιριάζει με το αναμενόμενο τερματικό, προχωρά στο επόμενο σύμβολο εισόδου
 - Για κάθε **μη τερματικό σύμβολο**, καλώ την αντίστοιχη συνάρτησή του
 - Π.χ. το **B → a B b** παράγει τον εξής κώδικα:

```
match( 'A_TOKEN' )  
B()  
match( 'B_TOKEN' )
```

Μέθοδος αναδρομικής κατάβασης

- Κάθε συνάρτηση υλοποιεί **όλους** τους **κανόνες** που έχουν στο αριστερό μέρος το αντίστοιχο μη τερματικό σύμβολο
 - Ένας **κλάδος if** για κάθε κανόνα
 - **Πώς διαλέγω**; στην απλή γραμματική του προηγούμενου παραδείγματος, αρκεί να ελέγχω το **επόμενο σύμβολο εισόδου** σε σχέση με το τερματικό σύμβολο που ξεκινά το δεξί μέρος κάθε εναλλακτικού κανόνα
 - Δεν αρκεί για πιο σύνθετες γραμματικές...

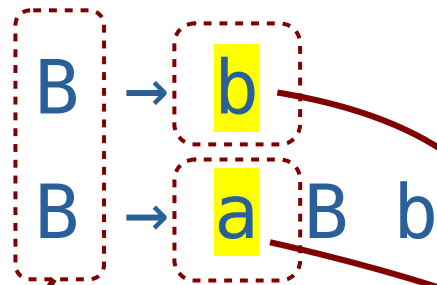
$B \rightarrow b$

$B \rightarrow a B b$



αν το **επόμενο σύμβολο** στην είσοδο είναι **a** επιλέγω τον δεύτερο κανόνα, αν είναι **b** τον πρώτο

Παράδειγμα για τους κανόνες του B



```
def B():  
    if next_token=='B_TOKEN':  
        # B -> b  
        match('B_TOKEN')  
  
    elif next_token=='A_TOKEN':  
        # B -> a B b  
        match('A_TOKEN')  
        B()  
        match('B_TOKEN')  
  
    else:  
        raise ParseError("...error msg...")
```


Η κλάση LL1ParserBase

- Περιλαμβάνεται στη βιβλιοθήκη του εργαστηρίου **compilerlabs**
 - Χρησιμοποιείται ως κλάση βάσης για την κατασκευή LL(1) parsers με τη μέθοδο της αναδρομικής κατάβασης (recursive descent)
 - Αρχικοποιείται με παράμετρο τον λεκτικό αναλυτή (**scanner**) που θα εξάγει τα tokens

```
from compilerlabs import LL1ParserBase
```

```
class MyParser(LL1ParserBase):
```

```
    def __init__(self, scanner):  
        super().__init__(scanner)
```

Η κλάση LL1ParserBase (2)

- Τι μας παρέχει
 - `self.next_symbol`: το επόμενο σύμβολο (token) στον πηγαίο κώδικα εισόδου
 - Όπως επιστρέφεται από τον λεκτικό αναλυτή
 - `self.match()`: η μέθοδος που εκτελεί τη λειτουργία `match` του συντακτικού αναλυτή LL(1)
- Τι πρέπει να προσθέσουμε
 - `self.parse()`: η μέθοδος που ξεκινά τη συντακτική ανάλυση

```
scanner = tokenizer.scan(text)
parser = MyParser(scanner)
parser.parse()
```

Η κλάση `ParseError`

- Επίσης στη βιβλιοθήκη `compilerlabs`
 - `ParseError`: ένα γενικό σφάλμα συντακτικής ανάλυσης που μπορούμε να χρησιμοποιήσουμε στον κώδικά μας

```
from compilerlabs import ParseError
```

- Παράδειγμα δημιουργίας σφάλματος

```
raise ParseError(f'Expecting A_TOKEN, found  
{self.next_symbol.token} instead')
```

- Παράδειγμα αναγνώρισης σφάλματος

```
try:  
    parser.parse()  
  
except ParseError as e:  
    print(e)
```